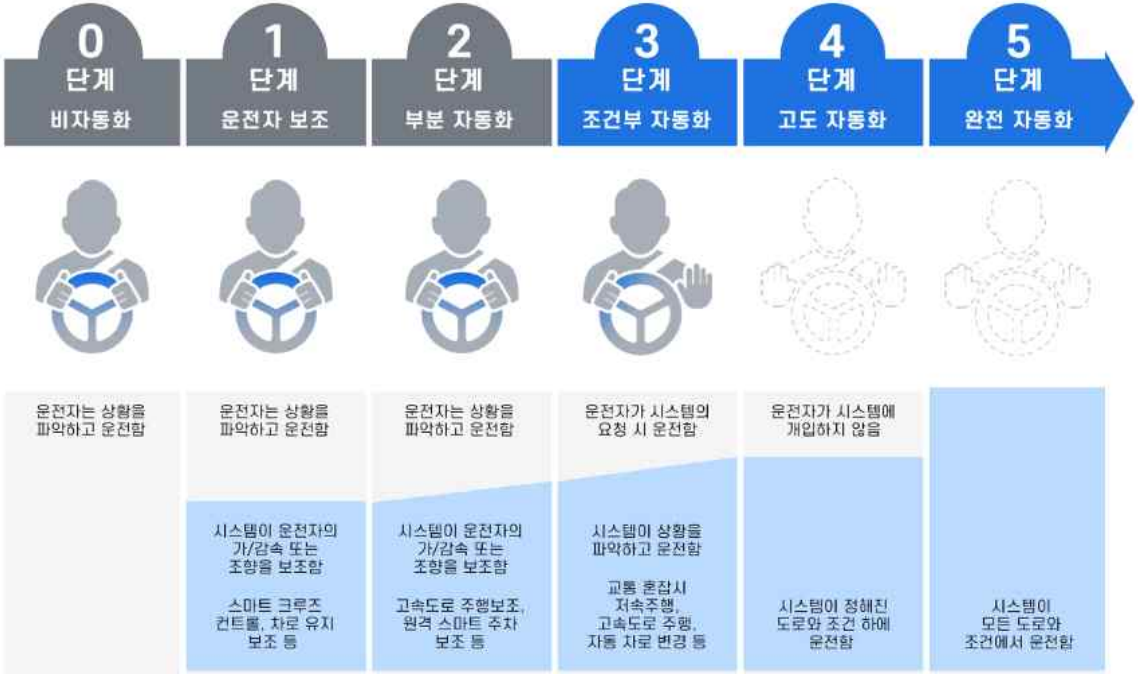
SW개발/HW제작 설계서

프로젝트 명 : Vision AI를 활용한 교차로 교통사고 방지 시스템

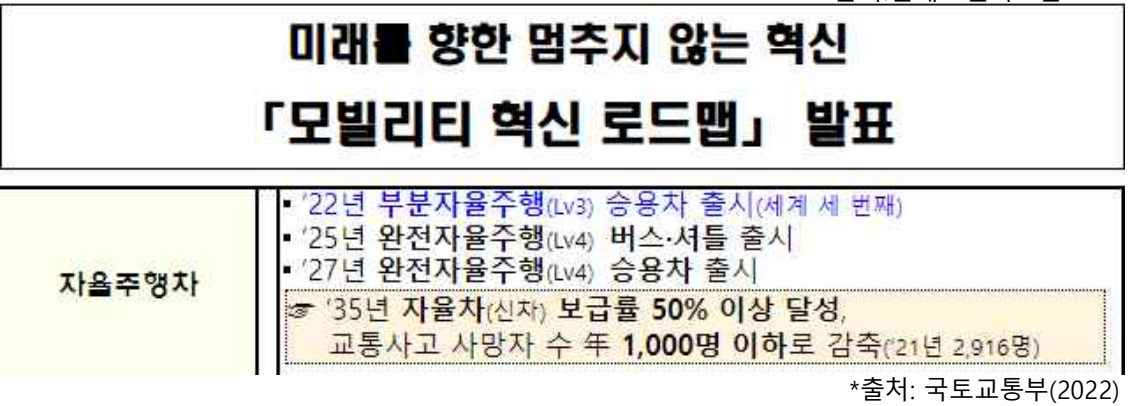
2024. 09. 02

(팀명)

| 시장/기술 동향 분석



*출처:현대트랜시스블로그



*출처: 국토교통부(2022)

- 운전자의 개입이 없는 '완전자율주행 5단계'가 실현되려면 많은 기술력과 연구 개발 시간이 필요하다.

- 현재 2~3단계에 머물어 있고, 국토 교통부의 '모빌리티 혁신 로드맵'에 따르면 27년 '완전자율주행 4단계' 승용차를 출시한다고 했지만, 지금 많은 기술적/정책적 한계에 부딪힌 상황이다.

- 따라서 자율주행차가 자체 센서로 인지한 정보뿐만 아니라 도로 주변에 설치된 인프라 등을 통해 정보를 받아 주행에 활용하는 '자율협력주행' 기술을 개발하고 있다.

시장/기술 동향 분석

ISSN 2383-6318(Print) / ISSN 2383-6326(Online)
KIISE Transactions on Computing Practices, Vol. 27, No. 8, pp. 394-399, 2021. 8
https://doi.org/10.5626/KTCP.2021.27.8.394

엣지카메라기반 실시간 도로위험 검지 및 정보 제공 시스템

(Edge Camera based Real-time Traffic Hazard Detection and Information Provision System)

백 장 운 [†] 이 준 구 [††] 최 윤 원 [††] 임 길 택 [††]
(Jang Woon Baek) (Joon-Goo Lee) (Yun Won Choi) (Kil Taek Lim)

요 약 현재 C-ITS 서비스는 레이다 혹 카메라 기반으로 돌발상황을 검지하고 도로위험 정보를 운전자에게 제공한다. 레이다기반의 돌발상황검지기는 가격이 비싸고 교통 이상 상황을 확인하기 위해서는 별도의 카메라가 필요하다. 또한, 레이다는 차량과 보행자 검출이 가능하나 그 외 오토바이, 자전거, 낙하물과 같은 객체는 검출이 어렵다. 기존 C-ITS 시스템에서 카메라기반의 돌발상황검지의 경우, C-ITS 센터의 영상분석 서버에서 돌발상황을 검지하고 위험정보를 운전자에게 제공하기 때문에 실시간 서비스를 제공하기 어렵다. 이러한 문제를 해결하기 위해 본 논문은 엣지카메라기반 실시간 도로위험인식 및 정보제공 시스템을 제안한다. 엣지카메라는 로컬에서 영상처리를 통해 객체를 인식하고 위험상황을 판단하여 RSU를 통해 운전자에게 위험정보를 제공한다. 제안 시스템은 실시간 처리를 위해 경량 딥러닝 네트워크를 사용하고, 정확도 향상을 위해 지역최적화를 수행한다. 그리고 기존 C-ITS 응용 프로그램을 수정 없이 사용할 수 있도록 엣지카메라와 RSU의 통신 인터페이스를 설계하였다. 실험을 통해 교통객체를 정확하게 인식하고 100ms 이내에 실시간으로 도로위험상황을 운전자에게 정보를 제공함을 확인할 수 있다.

키워드: 엣지카메라, 지역최적화, 도로위험검지, 돌발상황검지, C-ITS 서비스

ISSN 2383-6318(Print) / ISSN 2383-6326(Online)
KIISE Transactions on Computing Practices, Vol. 29, No. 6, pp. 289-295, 2023. 6
https://doi.org/10.5626/KTCP.2023.29.6.289

CCTV 영상에서 단일 및 다중 보행자의 횡단 의도 예측을 위한 데이터셋 및 모델

(A Dataset and Model for Single- and Multi-Pedestrian Crossing Intention Prediction in Surveillance Video)

함 제 석 [†] 김 보 경 [††] 문 진 영 [†††]
(Je-Seok Ham) (Bo Gyeong Kim) (Jinyoung Moon)

요 약 횡단보도, 차도, 또는 인도 위를 보행하는 보행자의 안전을 위해서 보행자의 횡단을 사전에 예측하는 보행자 횡단 의도 예측 기술의 중요성이 점차 증대되고 있다. 현재까지 대부분의 연구자들은 자동차에 설치된 온보드 카메라 영상을 기반으로 데이터셋을 구축하고, 이를 활용해 보행자의 횡단 의도를 예측하는 다양한 모델들을 개발하였다. 본 논문에서는 CCTV 영상을 기반으로 보행자 횡단 의도 예측 모델 개발을 위해 구축한 School zone Human Action (SHA) 데이터셋을 소개한다. 또한, 단일 및 다중 보행자의 횡단 의도 예측을 위한 베이스라인 모델을 제안한다. 본 CCTV 영상에서의 단일 및 다중 보행자 횡단 의도 예측 기술은 스마트 횡단 보도에서 보행자 횡단 의도 예측 결과에 따라 사전에 운전자에게 주의를 요구하거나 보행자에게 경고음을 알리는 시스템과 같은 응용 서비스에 적용 가능하다.

키워드: 보행자 횡단 의도 예측, SHA 데이터셋, 단일 보행자 행동 예측, 다중 보행자 행동 예측, 스마트 횡단 보도

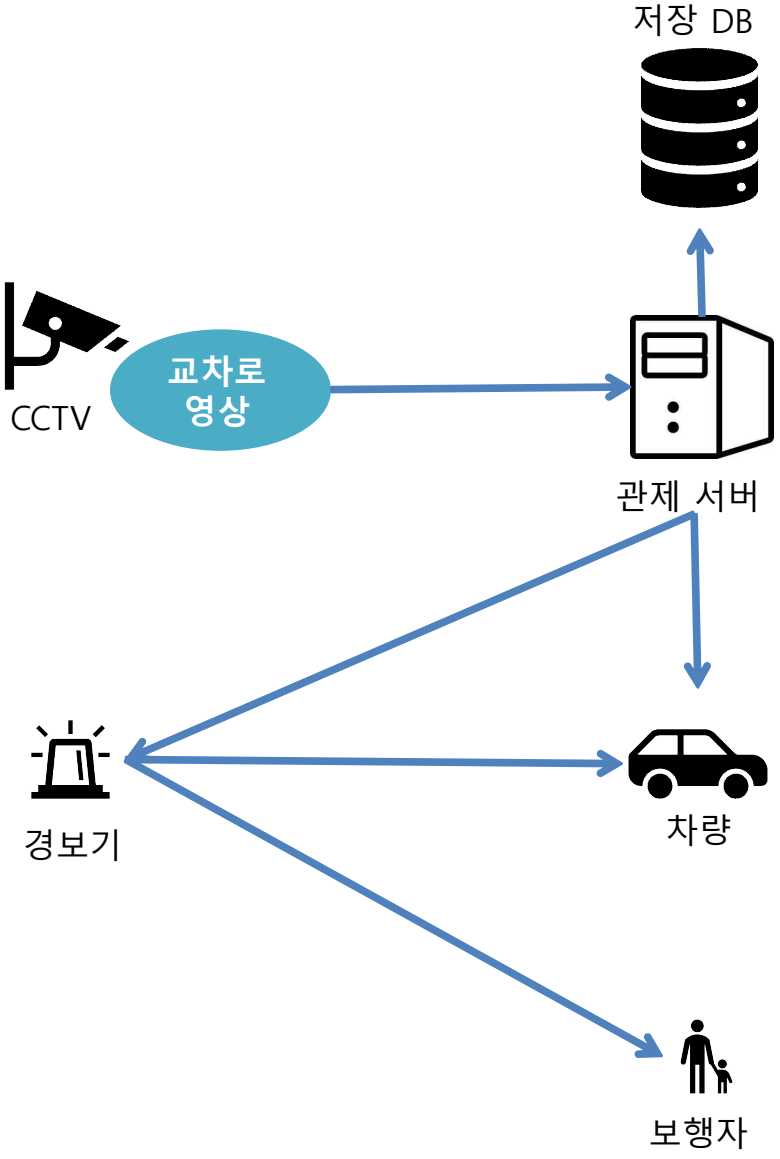
기존에 비슷한 연구가 있었지만, 차량을 실제로 제어하며 직접 교통 흐름을 컨트롤 하는 연구는 없었다.

| 요구사항 정의서

구분	기능	설명
S/W	객체 탐지	YOLO를 활용하여 카메라를 통해 객체를 탐지할 수 있다.
	객체 추적	YOLO의 BoT-Sort를 활용하여 카메라를 통해 객체를 추적할 수 있다.
	도로 위 관심 영역 설정	객체가 탐지되는 도로 위 영역을 OpenCV를 활용하여 설정할 수 있다.
	객체 위치 추정	Bird's Eye View Transformation을 활용하여 도로 위 객체의 위치를 2차원 영상 좌표계에 표현할 수 있다.
	차량 속도 추정	Bird's Eye View Transformation을 활용하여 나온 차량의 위치값을 통해 차량의 속도를 추정할 수 있다.
	차량 속도 및 방향 제어	차량 제어 파이썬 코드를 구성하여 차량의 속도와 방향을 제어할 수 있다.
	차선 인식 주행	OpenCV를 통해 차선을 인식하고 그에 따라 차량이 주행할 수 있다.
	V2X통신	도로 위 상황에 따라 서버와 차량 간 데이터를 송수신할 수 있다.
	모니터링	PyQt를 통해 교차로 상황 정보를 모니터링 할 수 있다.
	아두이노 위험 상황 경보기	아두이노를 통해 LED, 경보기를 제어할 수 있다.
	데이터 베이스	관제 센터에서 처리된 각종 객체 정보와 위험도, 시간 등이 데이터베이스에 저장할 수 있다.

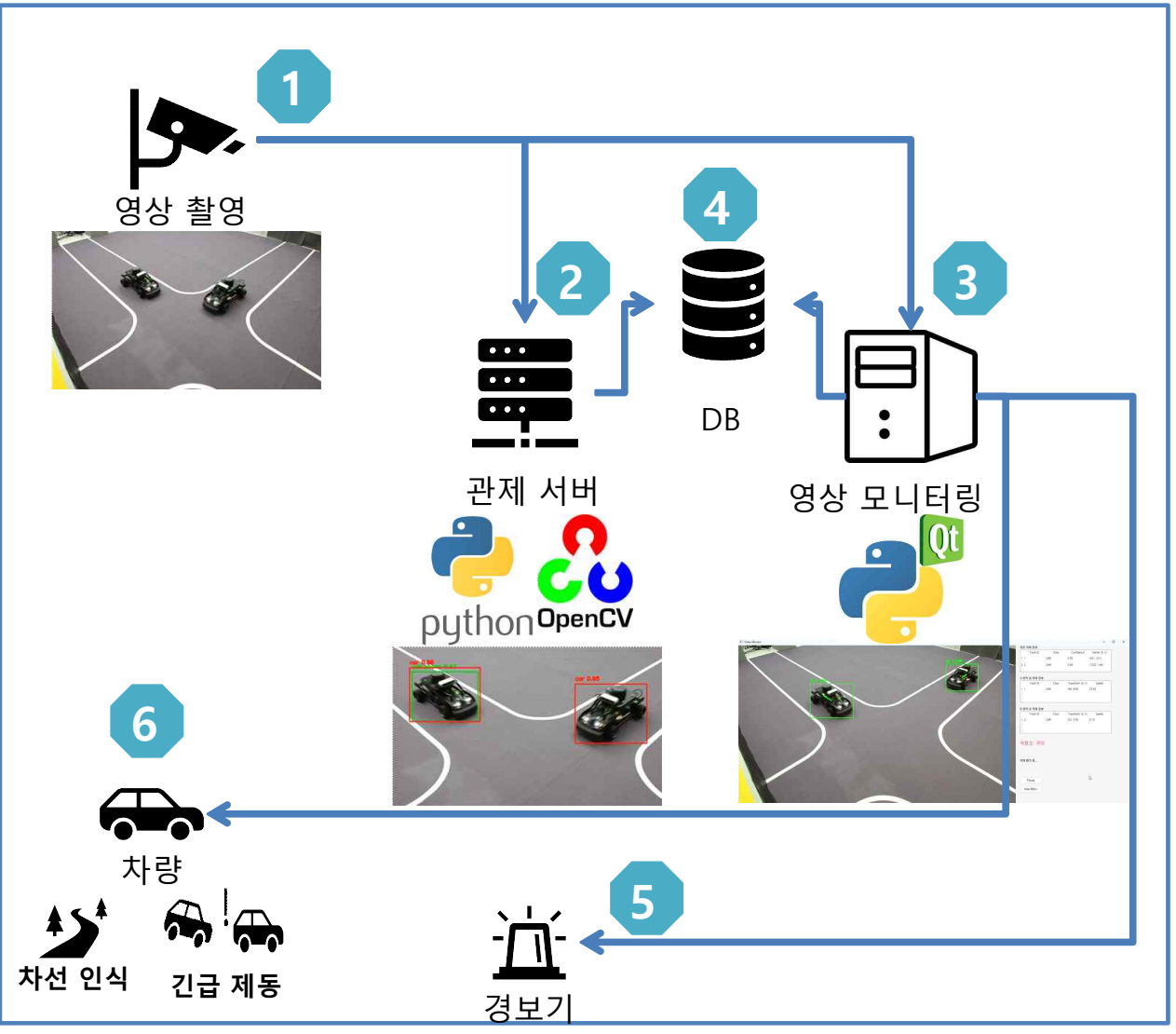
구분	기능	설명
H/W	차량 속도 및 방향 제어	차선인식 주행을 하며 차량의 속도와 방향을 조절할 수 있다.
	아두이노 위험 상황 경보기	위험 상황에 따라 LED를 표시하고, 경보를 울려 보행자와 차량에게 상황을 알려줄 수 있다.

| 유즈케이스



항목	설명		
개요	사용자가 영상을 통해 교차로 환경을 모니터링할 수 있다.		
관련 엑터	주 엑터	관제 서버(관제 센터)	
	보조 엑터	CCTV, 데이터베이스, 차량, 경보기, 보행자	
우선순위	상	중요도	3
		난이도	교차로 환경 속 상황을 모니터링하여 교차로 정보 및 시각화 자료를 제공할 수 있는 개발 능력
선행 조건	CCTV를 통해 교차로 환경을 파악하고 인공지능을 통한 상황 인지 및 분석 가능		
후행 조건	관련 데이터들이 실시간으로 관제 서버와 주고 받을 수 있어야 함		
시나리오	기본 시나리오	1. **CCTV**를 통해 교차로 환경을 촬영하고, 관제서버로 영상을 송출한다. 2. **관제 서버**는 CCTV 화면 영상 속 객체의 정보들을 통해 위험 상황을 판단하고, 교차로 상황을 모니터링한다. 3. **관제 서버**는 판단된 위험 상황에 따라 경보기와 차량을 제어한다. 4. **데이터베이스**는 관제 서버에서 처리되는 영상, 객체 정보들을 저장하고 기록한다. 5. **관제 서버**는 데이터베이스에 저장된 영상과 정보들을 통해 향후 교차로 상황에 대한 기록 자료로 활용한다. 6. **경보기**는 관제 서버에서 받은 정보를 통해 교차로 상황을 차량과 보행자에게 전달한다.	
	대안 시나리오	1. 관제 서버에서 객체 인식 불가시 1) 객체 인식 및 추적 인공지능 모델을 다시 학습한다. 2) 재 학습된 모델을 통해 객체 인식 및 추적이 원활히 이루어 질 수 있도록 한다. 2. CCTV 정보가 관제 서버로 전송되지 않을 때 1) 통신망 혹은 네트워크 연결을 확인한다. 2) 프로그램을 재실행하여 실시간 정보 교환이 이루어 질 수 있도록 한다.	
비기능적 요구사항	성능: 차량과 보행자는 관제서버로 판단된 교차로 상황을 통해 움직임을 제어할 수 있어야 한다. 보안: 관제 서버와 데이터베이스, 경보기에는 관제 센터 외에는 접근할 수 없다.		

| 서비스 구성도 - 서비스 시나리오



1. CCTV
1) 교차로 환경을 촬영하여 관제 서버에 전송

2. 관제 서버
1) CCTV에서 받은 영상을 YOLO를 활용하여 객체 추적 실행
2) Python, OpenCV를 활용하여 영상 분석을 통해 위험도 판단 진행

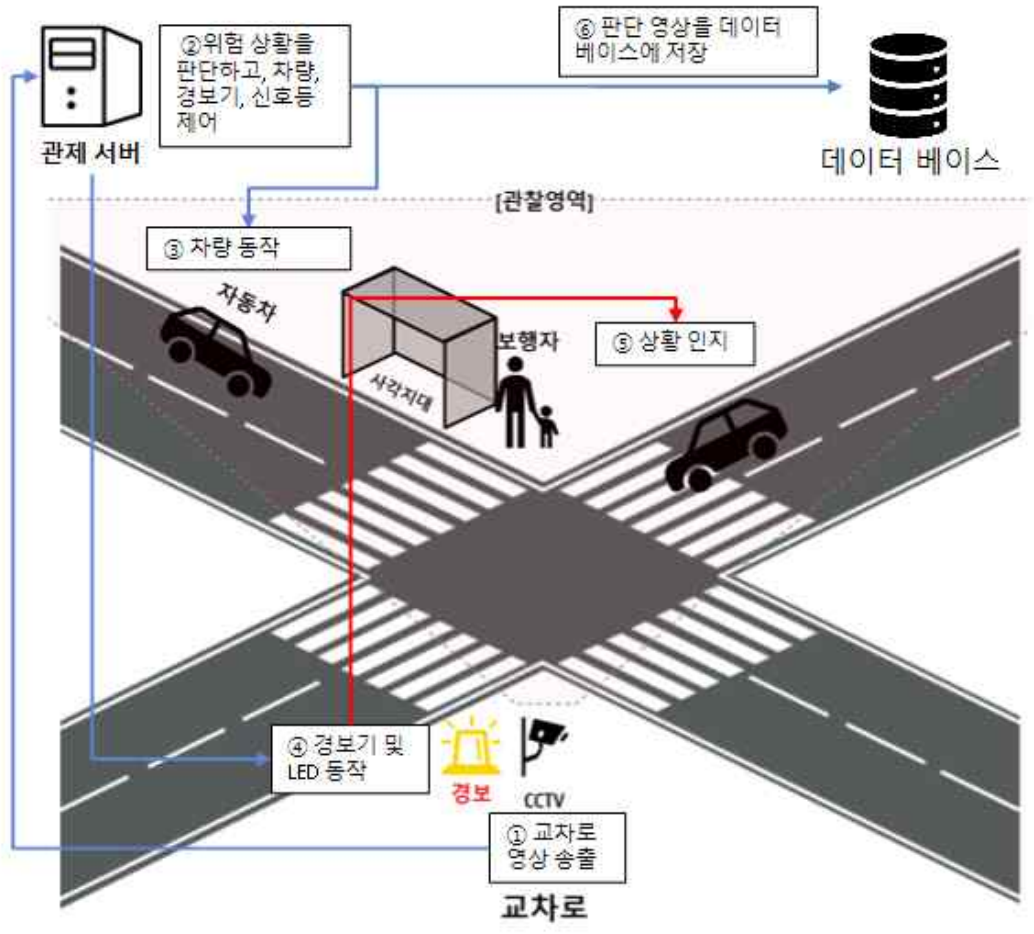
3. 영상 모니터링
1) Python PyQt를 활용하여 모니터링 진행

4. DB
1) 차량 객체 정보 및 위험도를 시간별로 데이터베이스에 저장

5. 위험 상황 경보기
1) 관제 서버에서 받은 상황 정보를 통해 동작

6. 차량
1) 차선 인식 주행, 긴급 제동 시스템 작동
2) 관제 서버에서 받은 상황 정보를 통해 감속 및 긴급 제동

| 서비스 흐름도



1. CCTV에서 교차로 영상을 촬영하여 관제 서버에 영상 송출

2. CCTV에서 받은 영상에서 객체 정보들을 추출하고, 위험 상황을 판단하여 위험 정보를 클라이언트(차량, 경보기)에 전송

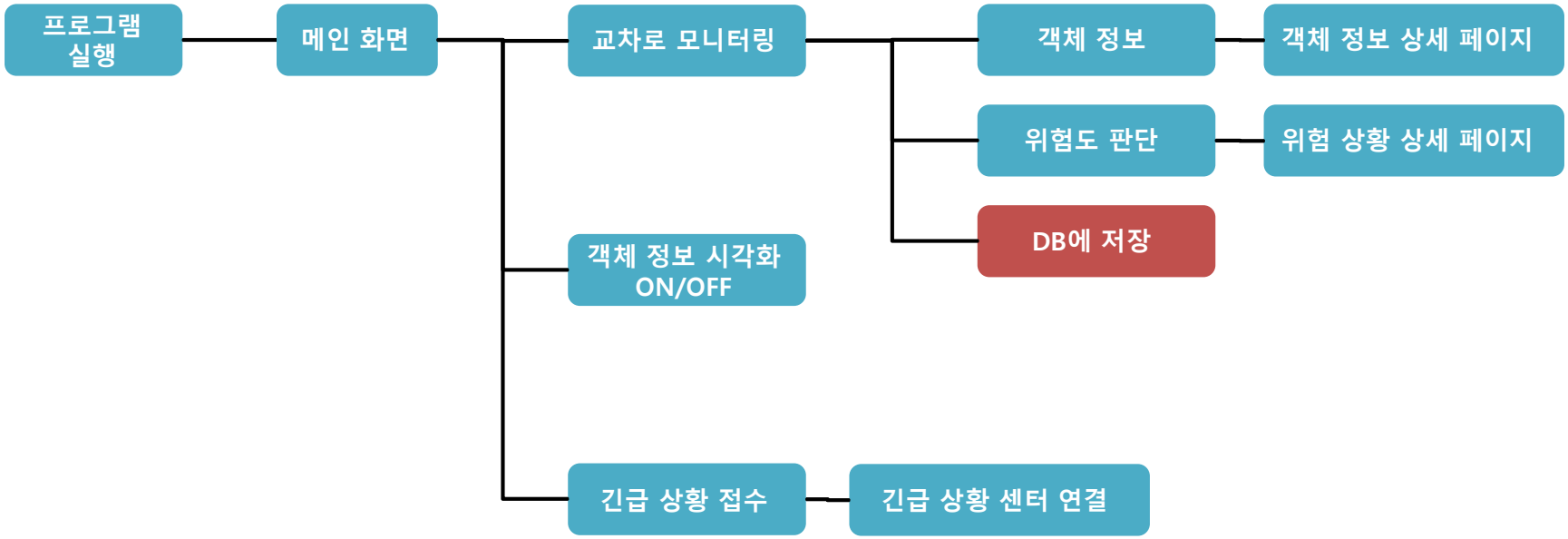
3. 서버에서 판단한 위험 상황에 따라 차량 제어

4. 서버에서 판단한 위험도에 따라 위험 상황 경보기 동작

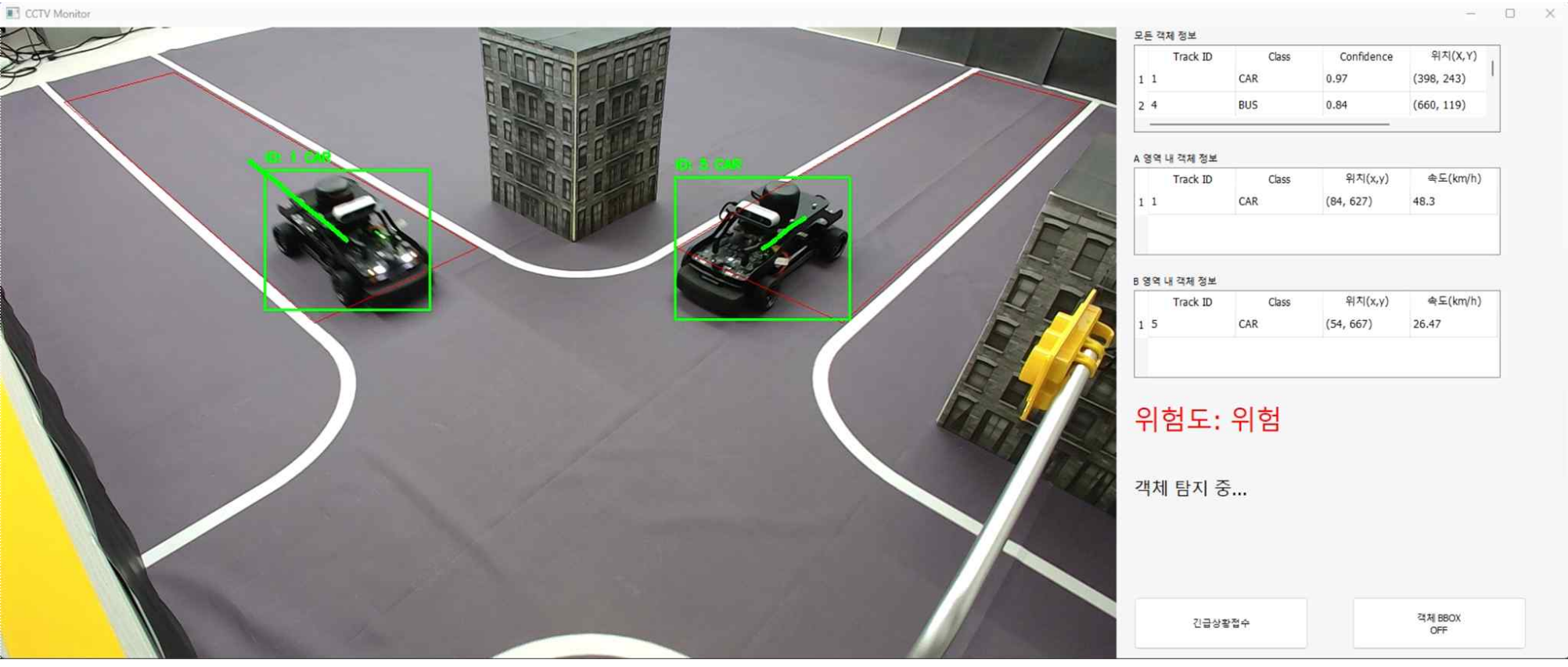
5. 경보기를 통해 보행자, 차량이 교차로 상황 인지

6. 데이터 베이스에 객체 정보 및 판단된 위험도, 영상을 저장

| 메뉴 구성도


프로그램 실행
메인 화면
교차로 모니터링
객체 정보
객체 정보 상세 페이지
위험도 판단
위험 상황 상세 페이지
DB에 저장
객체 정보 시각화 ON/OFF
긴급 상황 접수
긴급 상황 센터 연결

| 화면 설계서

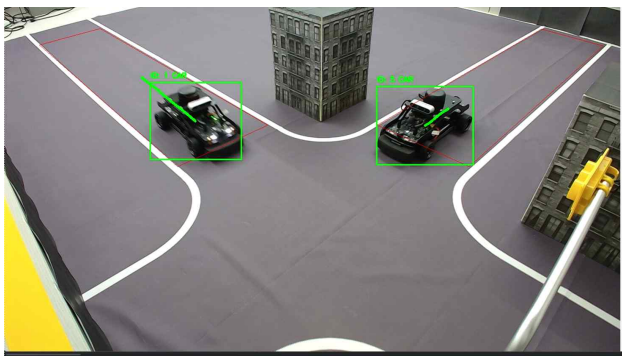

CCTV Monitor
ID: 1 CAR
ID: 3 CAR
모든 객체 정보
Track ID Class Confidence 위치(X,Y)
1 1 CAR 0.97 (398, 243)
2 4 BUS 0.84 (660, 119)
A 영역 내 객체 정보
Track ID Class 위치(x,y) 속도(km/h)
1 1 CAR (84, 627) 48.3
B 영역 내 객체 정보
Track ID Class 위치(x,y) 속도(km/h)
1 5 CAR (54, 667) 26.47
위험도: 위험
객체 탐지 중...
긴급상황접수
객체 BBOX OFF

| 화면 설계서

▶ 기능정보

기능명	실시간 교차로 상황
기능설명	CCTV 영상과 관제 서버에서 처리된 객체의 정보를 실시간으로 화면에 모니터링한다.
처리내용	• 실시간으로 추적되는 동적 객체를 표시한다. • 동적 객체를 추적하며 ID를 부여하여 같은 객체임을 판단하고, 이동 경로를 화면에 표시한다. • 객체가 차량인지 보행자인지 등에 대한 정보를 화면에 표시한다.

▶ 기술 적용 예시 정보



설명	교차로에서 실시간으로 차량이 영역 A(왼쪽 도로)에서 약 50km/h로 주행하고 있고, 영역 B(오른쪽 도로)에서 약 30km/h로 주행 중인 상황의 영상이다.
기술 적용	• 추적되는 동적 객체의 BBoX를 화면에 표시한다. • 객체가 CAR인지, PERSON인지 등에 대한 정보와 동일 객체에 대한 추적을 시각화 하기 위해 ID를 표시한다. • 동적 동일 객체를 추적하면서 ID를 부여하고, 그 이동 경로를 시각화 한다.

| 화면 설계서

▶ 기능정보

기능명	실시간 교차로 정보
기능설명	현재 교차로 속 객체의 정보(ID, 클래스, 신뢰도, 위치, 속도)와 위험도를 나타낸다.
처리내용	• 교차로 속 객체들에 대한 상세한 정보(ID, 클래스, 신뢰도, 위치, 속도)를 나타낸다. • 판단된 위험도(안전, 주의, 위험)를 나타낸다.

▶ 기술 적용 예시 정보



설명	'모든 객체 정보' 표에서는 탐지된 모든 객체의 정보를 출력하고, 'A 영역 내 객체 정보'와 'B 영역 내 객체 정보'는 지정한 도로 위 영역에 따라 객체의 정보를 분리하여 출력된다. 해당 정보들을 통해 위험도를 판단하고, 판단된 결과(위험, 주의, 안전)는 빨간 글씨로 표시된다.
기술 적용	• PyQt의 QTableWidget()을 통해 객체 정보를 표에 출력한다. • 판단된 위험 상황 결과(위험, 주의, 안전)를 표시한다.

| 화면 설계서

▶ 기능정보

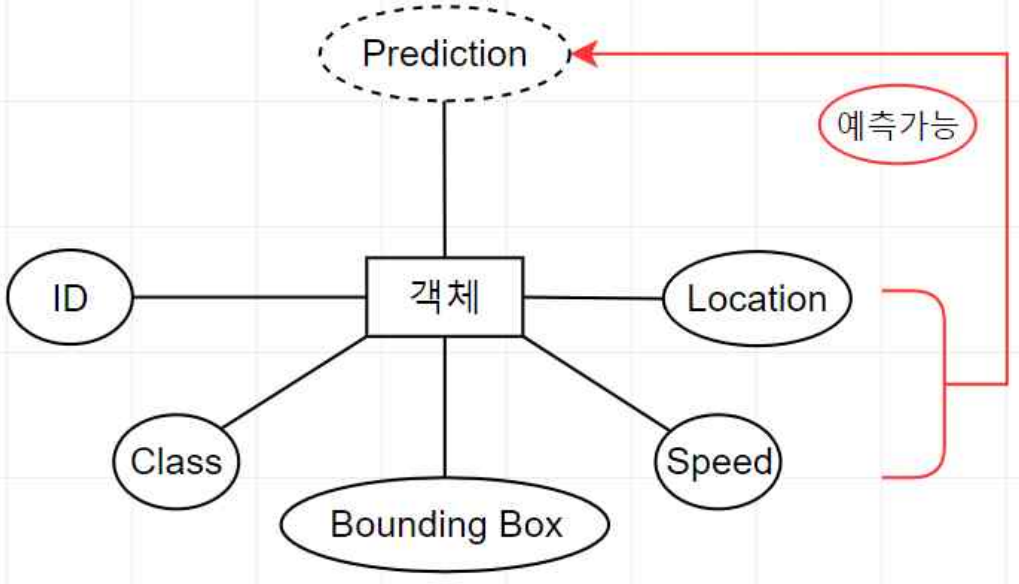
기능명	도구
기능설명	객체 정보 시각화 및 긴급 상황 접수
처리내용	• 도구 창에서 '객체 BBOX ON/OFF' 버튼을 눌러 객체에 대한 시각 정보를 제어한다. • '긴급상황접수' 버튼을 눌러 긴급 상황 센터와 정보를 공유한다.

▶ 기술 적용 예시 정보

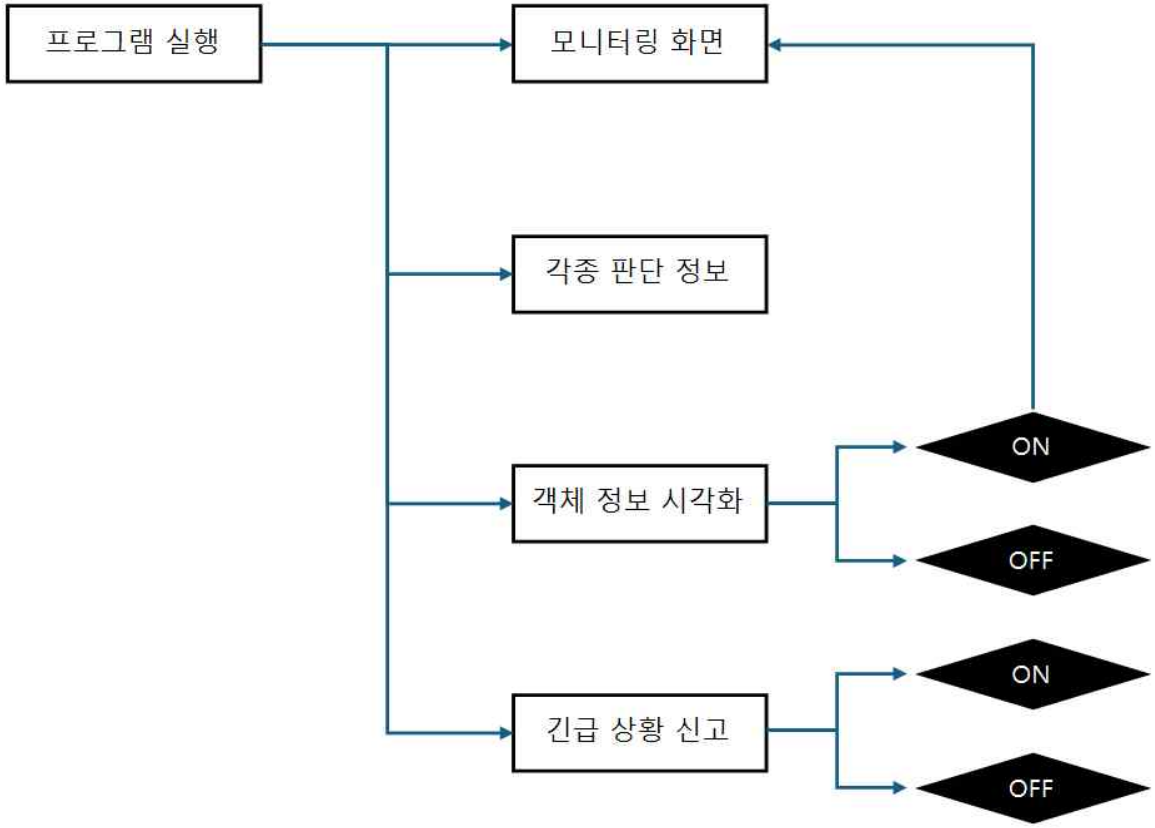
<table>
<tr><td rowspan="2"> 
</td><td>설명</td><td>실시간으로 추적되는 객체의 바운딩 박스에 대한 정보를 '객체 BBOX On/OFF 버튼을 눌러 화면에 출력, 미출력하고 있다.</td></tr>
<tr><td>기술 적용</td><td>• 도구 창에서 '객체 BBOX ON/OFF' 버튼 클릭시 '실시간 교차로 상황' 창에 표시되는 객체 정보를 ON/OFF한다.
• 도구 창에서 '긴급상황접수' 버튼 클릭시 긴급 상황 센터와 정보를 공유한다.</td></tr>
</table>

| 엔티티관계도 - ERD

객체		
PK	ID	int(20)
	Class	bool()
	BoundingBox	float(20)
	Speed	int(20)
	Location	int(20)
	Prediction	float(20)

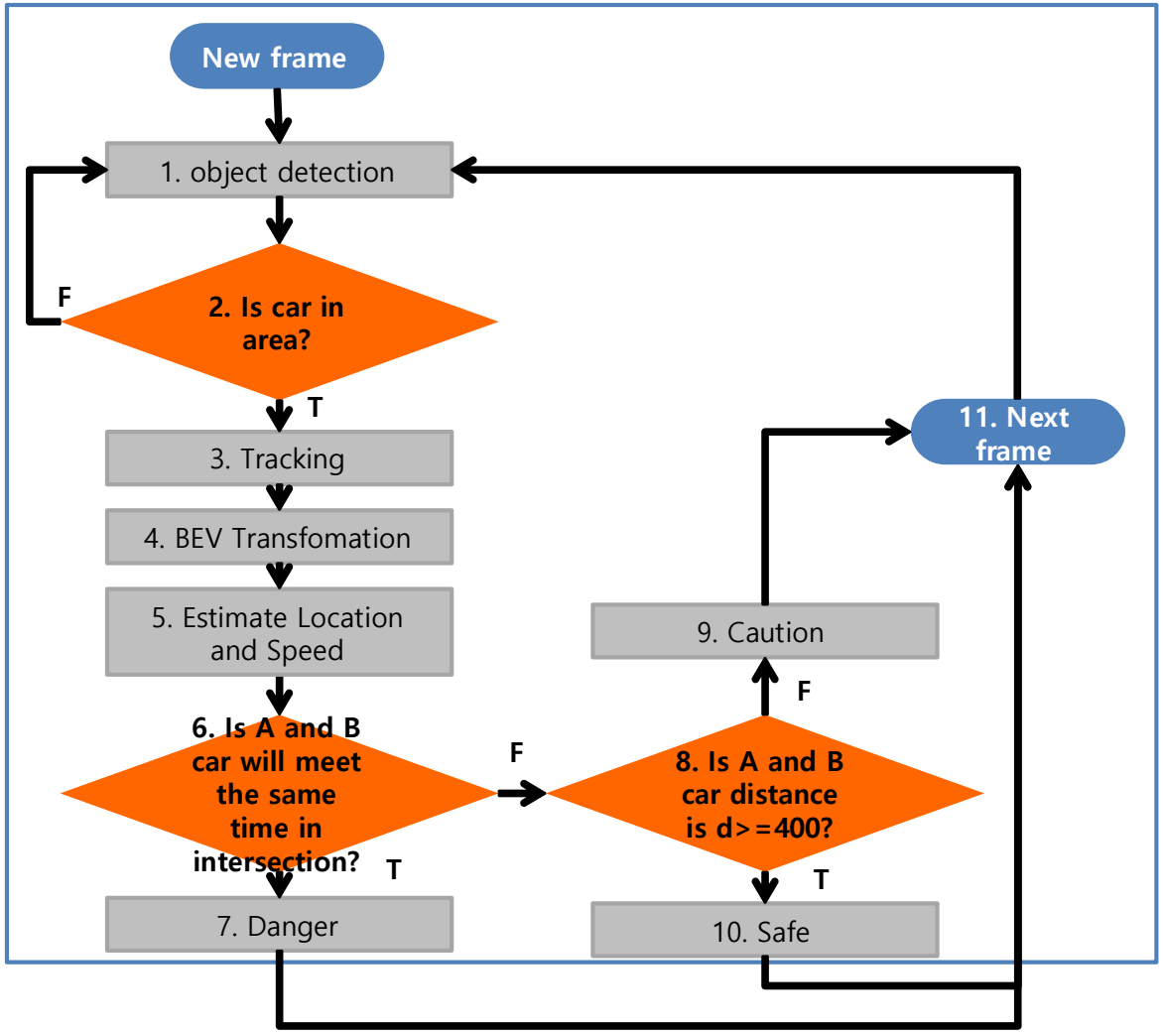


| 기능 처리도(기능 흐름도)



1. 모니터링 화면을 통해서 교차로 위험 판단 과정을 확인할 수 있다.
2. 모니터링 화면과 함께 각종 판단 정보(객체 정보(ID, Class, 위치, 속도 등), 위험도, 상태)을 확인할 수 있다.
3. 객체 정보 시각화를 통해서 모니터링 화면 내 객체에 대한 판단 정보를 켜고 끌 수 있다.
4. 모니터링 중 위험 상황이 생기면 긴급 상황 신고를 눌러 긴급 상황 센터와 연결된다.

| 알고리즘 명세서

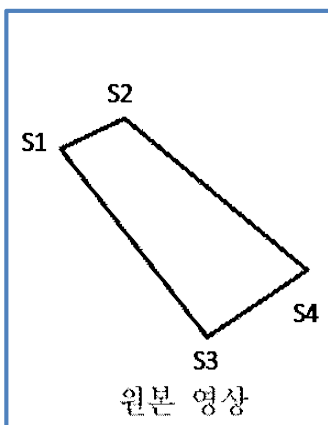


위험 상황 판단 알고리즘 시나리오

1. 영상을 받아서 YOLO를 통해 객체를 인식한다.
2. 객체가 지정한 영역 안에 들어오지 않으면 다시 객체 인식을 한다.
3. 객체가 영역 내부에 들어오면 인식된 객체를 추적한다.
4. 추적된 객체의 특징값을 BEV 변환을 이용하여 왜곡 문제와 차원 간의 차이를 최소화한다.
5. 변환된 특징값으로 객체의 위치와 속도를 측정한다.
6. 속도를 통해 위치를 예측하고, A 영역과 B 영역에 있는 차량이 같은 시간에 교차로에서 만나는지 파악한다.
7. 같은 시간에 만나면 상황을 위험으로 판단한다.
8. 예측한 위치의 값의 거리가 400 pixel 이상 차이가 나는지 파악한다.
9. 400 pixel 미만이면 상황을 주의로 판단한다.
10. 400 pixel 이상이면 상황을 안전으로 판단한다.
11. 상황이 판단된 경우 다시 객체를 인식하고 추적하여 반복하여 예측 값을 비교한다.

Bird's Eye View(BEV) Transformation - 위치 및 속도 추정 알고리즘

BEV 변환은 카메라로 인해 왜곡된 영상을 하늘에서 바라봤을 때 보이는 시점으로 영상을 처리한다. 이를 통해 멀리 있는 객체의 움직임이 작은 것을 해소하여 원근 왜곡 현상을 해소하고, 3차원 좌표계의 값을 2차원 영상에 표현할 수 있다. 매 프레임마다 추적된 객체의 특징값(중심 좌표)을 BEV 변환을 통해 변환된 특징값으로 처리하고, 처리된 값을 활용하여 객체와 교차로, 객체 간의 거리를 구한다. 또한 매 프레임마다 객체 중심 값의 변화량을 통하여 속도를 추정한다. 이를 통해 동적 객체의 위치와 속도를 매 프레임마다 확인할 수 있으며, 향후 객체의 위치를 예측하여 교차로에서 발생하는 충돌 사고를 미리 인지하여 방지할 수 있다. 만약 객체의 위치 예측값이 A 영역의 차량과 B 영역의 차량이 동일하다면 교차로에서 충돌하여 상황이 '위험'으로 판단된다. 하지만 동일하지 않다면 객체의 중심간의 거리를 재고, 일정 임계치보다 크다면 상황이 '안전', 일정 임계치보다 작다면 '주의'로 판단된다.



BEV

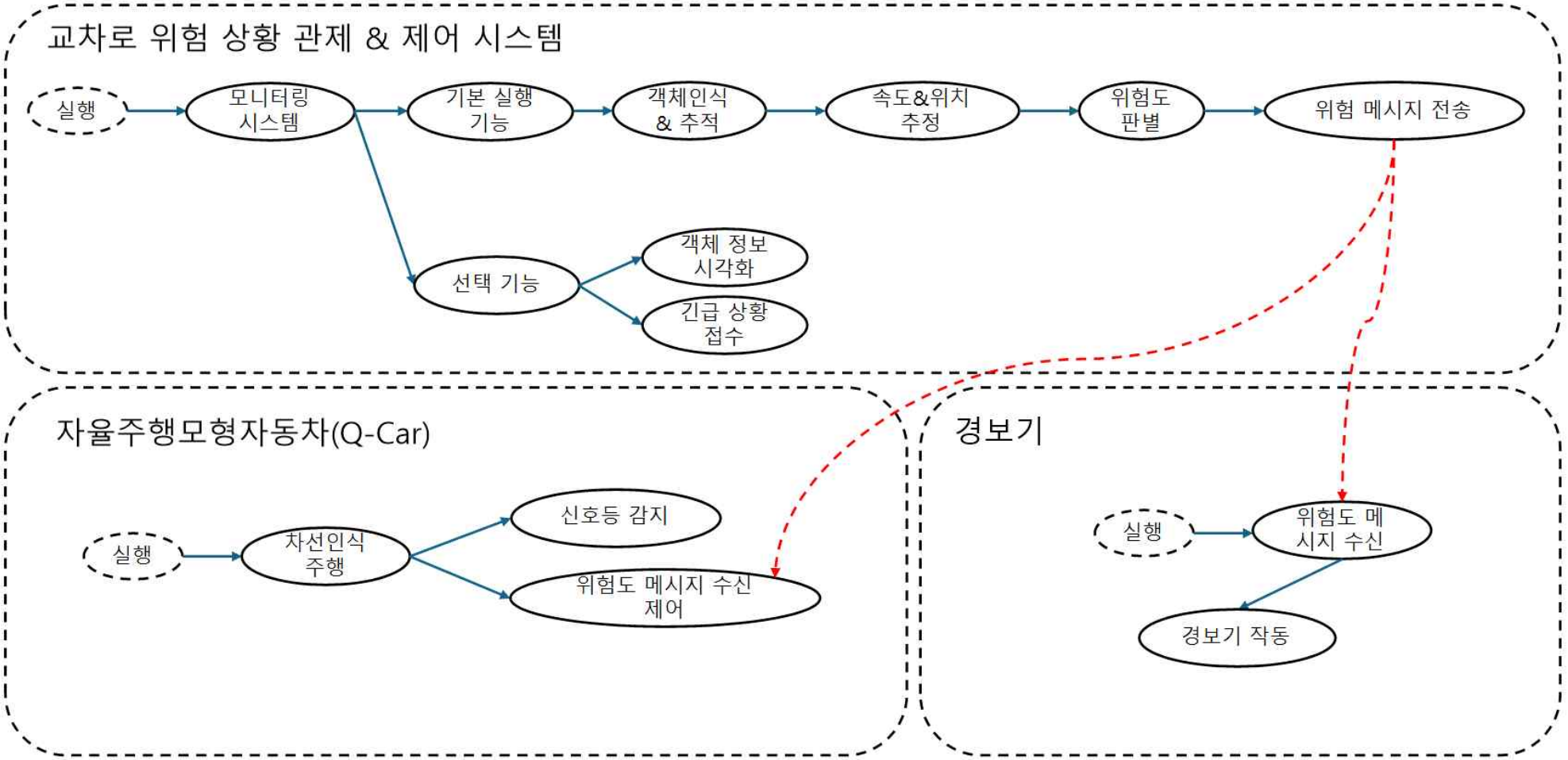




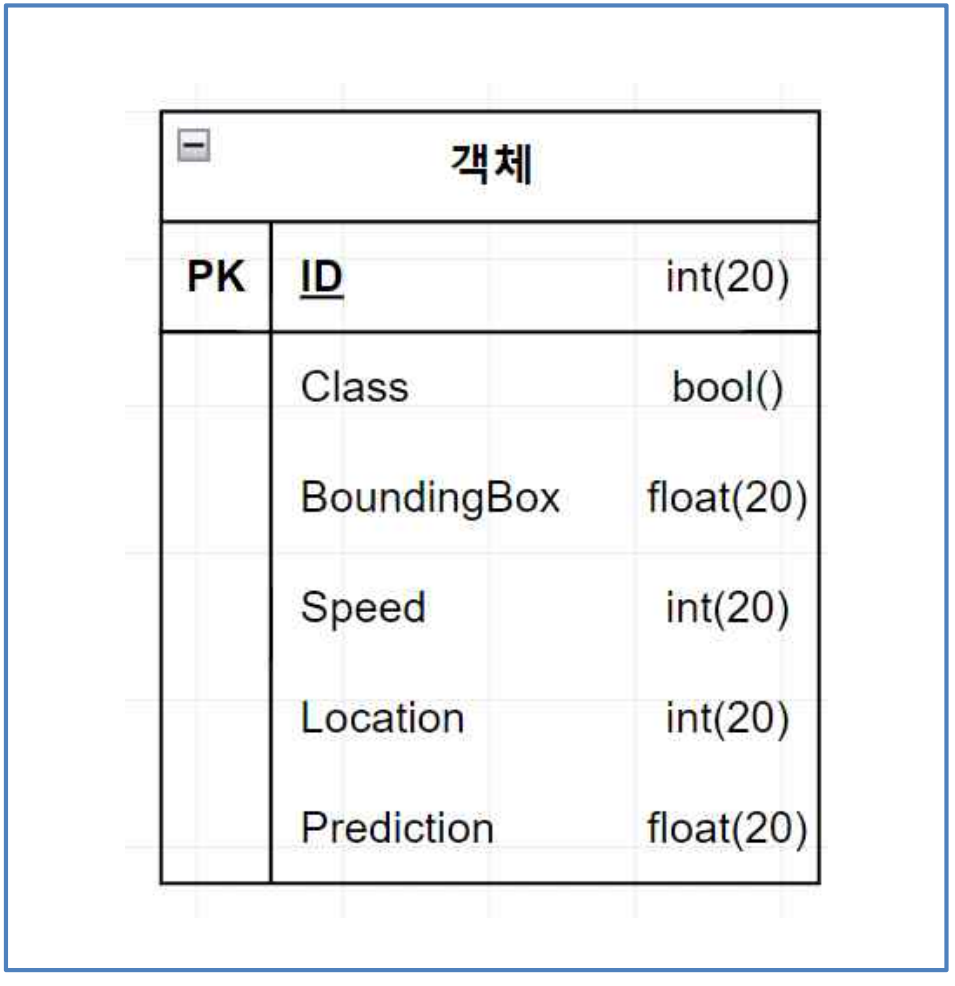
| 프로그램 - 목록

기능 분류	기능 명	기능 설명
CCTV 카메라	영상 촬영	교차로 환경 영상 실시간 촬영
모니터링 서버	객체 탐지	교차로 영역 내 객체 탐지
	객체 추적	교차로 영역 내 탐지된 객체 추적
	객체 위치 추정	BEV 변환 활용 객체 위치 추정
	객체 속도 추정	BEV 변환 활용 객체 속도 추정
	위험 상황 판단	객체 위치 예측을 통한 위험 상황 판단
	교차로 상황 영상	CCTV에서 받은 영상에 객체의 처리 정보가 담긴 영상
	도구	객체 정보 시각화, 긴급 상황 접수
CCTV Monitor	위험 및 판단 정보 확인	탐지된 교차로 정보와 판단된 위험도 모니터링
위험 상황 경보기	경보 및 LED 점등	위험 상황에 따라 경보 및 LED 점등
차량	차선 인식 주행	차선을 인식하여 차량을 주행
	전방 추돌 방지	Depth Camera를 이용한 전방 추돌 방지
	V2X 통신	소켓 통신으로 위험도를 전송 받아 차량 제어

| 데이터 수집처리 정의서


교차로 위험 상황 관제 & 제어 시스템
실행
모니터링 시스템
기본 실행 기능
객체인식 & 추적
속도&위치 추정
위험도 판별
위험 메시지 전송
선택 기능
객체 정보 시각화
긴급 상황 접수
자율주행모형자동차(Q-Car)
실행
차선인식 주행
신호등 감지
위험도 메시지 수신 제어
경보기
실행
위험도 메시지 수신
경보기 작동

| 테이블 정의서 - ERD



객체		
PK	ID	int(20)
	Class	bool()
	BoundingBox	float(20)
	Speed	int(20)
	Location	int(20)
	Prediction	float(20)

| 핵심소스코드(1)

YOLO 객체 탐지 및 추적 코드

```
def main():
    cv.namedWindow('T_TEST')
    cv.setMouseCallback( windowName: 'T_TEST', Mouse)
    cv.namedWindow('Trans')
    cv.setMouseCallback( windowName: 'Trans', Mouse)

    cap = cv.VideoCapture(0)
    cap.set(cv.CAP_PROP_FRAME_WIDTH,  value: 1280)
    cap.set(cv.CAP_PROP_FRAME_HEIGHT,  value: 736)
    cap.set(cv.CAP_PROP_FPS,  value: 60)

    model = YOLO("epoch52.pt").to(device)

    while True:
        ret, frame = cap.read()
        if ret:
            results = model.track(frame, conf=0.6, imgsz=(736, 1280), persist=True,
                                  verbose=False)
```

| 핵심소스코드(2)

BEV 변환 코드

```
class ViewTransformer:
    def __init__(self, source: np.ndarray, target: np.ndarray) -> None:
        source = source.astype(np.float32)
        target = target.astype(np.float32)
        self.m = cv.getPerspectiveTransform(source, target)
    def transform_points(self, points: np.ndarray) -> np.ndarray:
        if points.size == 0:
            return points
        reshaped_points = points.reshape(-1, 1, 2).astype(np.float32)
        transformed_points = cv.perspectiveTransform(reshaped_points, self.m)
        return transformed_points.reshape(-1, 2)
```

```
sourceA = np.array([[74,85], [200,51], [361,337], [548,251]]) # 2, 1, 3 ,4
sourceB = np.array([[1116,52], [1244,87], [775,251], [966,337]]) # 2, 1, 3 ,4
target = np.array([[0, 0], [T_W, 0], [0, T_H], [T_W, T_H]])

transformerA = ViewTransformer(source=sourceA, target=target)
transformerB = ViewTransformer(source=sourceB, target=target)

imgA = np.array( object: [[74,85], [200,51], [361,337], [548,251]], dtype=np.float32) # 2, 1, 3 ,4
imgB = np.array( object: [[1116,52], [1244,87], [775,251], [966,337]], dtype=np.float32) # 2, 1, 3, 4
trn = np.array( object: [[0, 0], [T_W, 0], [0, T_H], [T_W, T_H]], dtype=np.float32)

matrixA = cv.getPerspectiveTransform(imgA, trn)  # 원본창, 변환창
matrixB = cv.getPerspectiveTransform(imgB, trn)  # 원본창, 변환창
```

```
resultA = cv.warpPerspective(frame, matrixA,  dsize: (T_W, T_H))
resultB = cv.warpPerspective(frame, matrixB,  dsize: (T_W, T_H))
```

```
pointsTA = np.array([[x], [y]])
pointsTA = transformerA.transform_points(points=pointsTA).astype(int)
```

| 핵심소스코드(3)

객체 속도 추정

```
self.trackmA_history[track_id].append(transformed_pointA)
last_y = int(self.trackmA_history[track_id][-1][-1][1])
first_y = int(self.trackmA_history[track_id][0][0][1])
if last_y - first_y <= 0:
    self.A_v_list.clear()
    self.av = 0
else:
    self.A_v_list.append(int(transformed_pointA[0][-1]))
    self.A_slope = round(abs(self.A_v_list[-1] - self.A_v_list[0]) / len(self.A_v_list), 2)
    self.av = self.calculate_speed(self.A_slope)

self.current_a_position = transformed_pointA[0][-1]
self.current_a_speed = self.av
```

| 핵심소스코드(4)

객체 위치 추정

```
pointsTB = np.hstack(track_mB).astype(np.int32).reshape((-1, 1, 2))
cv.polylines(resultB, pts: [pointsTB], isClosed=False, color=WHITE, thickness=3)

pointsPB = np.hstack(predictB).astype(np.int32).reshape((-1, 1, 2))
pointsPB = np.array([[80, y] for y in pointsPB[:, 0, 1]], dtype=np.int32)
pointsPB = pointsPB.reshape((-1, 1, 2))
cv.polylines(resultB, pts: [pointsPB], isClosed=False, color=GREEN, thickness=3)


csv_B_py.append(pointsPB[-1][0][1])
b_slope = round(abs(csv_B_py[-1] - csv_B_py[0]) / len(csv_B_py),2)
B_slope = b_slope

dis = abs(pointsPB[0][0][1] - pointsPB[-1][0][1])
if Byp1 != -1:
    Byp1 = pointsPB[-1][0][1] + dis
if Byp2 != -1:
    Byp2 = pointsPB[-1][0][1] + dis * 2
if Byp3 != -1:
    Byp3 = pointsPB[-1][0][1] + dis * 3


cv.circle(resultB, center: (80, Byp1), radius: 5, RED, -1)
cv.circle(resultB, center: (80, Byp2), radius: 5, YELLOW, -1)
cv.circle(resultB, center: (80, Byp3), radius: 5, GREEN, -1)
```

| 핵심소스코드(5)

소켓 통신

```
import socket, time
import select

cnt = 0
host = '192.168.0.39'
port = 8878

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((host, port))
server_socket.listen()

print('echo server start')

# 클라이언트 소켓 리스트
client_sockets = [server_socket]
```

```
while True:
    # select를 사용하여 여러 소켓의 상태를 감시
    readable, _, _ = select.select(client_sockets, [], [])

    for readable_socket in readable:
        if readable_socket == server_socket:
            # 새로운 클라이언트가 접속한 경우
            client_soc, addr = server_socket.accept()
            client_sockets.append(client_soc)
            print(f'connected client addr: {addr}')
            cnt = cnt + 1
            print(cnt)
        else:
            # 기존 클라이언트로부터 데이터를 받은 경우
            data = readable_socket.recv(1024)
```

| 핵심소스코드(6)

PyQt 모니터링

```
# Video display widget
self.video_label = QLabel(self)
self.video_label.setFixedSize(1280, 720)

# Play/Pause button
self.is_paused = False
self.play_pause_button = QPushButton("긴급상황접수", self)
self.play_pause_button.setGeometry(1300, 650, 200, 60)
self.play_pause_button.clicked.connect(self.toggle_play_pause)

# BBox와 객체 정보 표시 토글 버튼
self.show_bbox = True
self.toggle_button = QPushButton("객체 BBOX\nOFF", self)
self.toggle_button.setGeometry(1550, 650, 200, 60)
self.toggle_button.clicked.connect(self.toggle_display)

self.text_table_widget = QLabel(self)
self.text_table_widget.setGeometry(1300, 0, 300, 20)
self.text_table_widget.setText("모든 객체 정보")

# Table widget to display object information
self.table_widget = QTableWidget(self)
self.table_widget.setColumnCount(4)
self.table_widget.setHorizontalHeaderLabels(["Track ID", "Class", "Confidence", "위치(X,Y)"])
self.table_widget.setGeometry(1300, 20, 420, 100)

self.text_a_area_table_widget = QLabel(self)
self.text_a_area_table_widget.setGeometry(1300, 140, 300, 20)
self.text_a_area_table_widget.setText("A 영역 내 객체 정보")

# A영역에 있는 객체 정보 테이블 위젯 (두 번째 표)
self.a_area_table_widget = QTableWidget(self)
self.a_area_table_widget.setColumnCount(4)
self.a_area_table_widget.setHorizontalHeaderLabels(["Track ID", "Class", "위치(x,y)", "속도(km/h)"])
self.a_area_table_widget.setGeometry(1300, 160, 420, 100)
```

```
self.text_b_area_table_widget = QLabel(self)
self.text_b_area_table_widget.setGeometry(1300, 280, 300, 20)
self.text_b_area_table_widget.setText("B 영역 내 객체 정보")

# B영역에 있는 객체 정보 테이블 위젯 (세 번째 표)
self.b_area_table_widget = QTableWidget(self)
self.b_area_table_widget.setColumnCount(4)
self.b_area_table_widget.setHorizontalHeaderLabels(["Track ID", "Class", "위치(x,y)", "속도(km/h)"])
self.b_area_table_widget.setGeometry(1300, 300, 420, 100)

# 충돌 예측 라벨
self.collision_label = QLabel(self)
self.collision_label.setGeometry(1300, 420, 500, 50)
self.collision_label.setStyleSheet("font-size: 30px; color: red;")
self.collision_label.setText("위험도: 안전")

# 텍스트 디스플레이 라벨
self.text_label = QLabel(self)
self.text_label.setGeometry(1300, 500, 500, 50)
self.text_label.setStyleSheet("font-size: 20px;")
self.text_label.setText("객체 탐지 중...")

# Timer setup for frame updates
self.timer = QTimer(self)
self.timer.timeout.connect(self.update_frame)
self.timer.start(30)

# 마지막 프레임 저장
self.last_frame = None
self.last_results = None
```

| 핵심소스코드(7)

PyQt 모니터링(버튼)

```
1 usage
def toggle_play_pause(self):
    if self.is_paused:
        self.timer.start(0)
        self.play_pause_button.setText("긴급상황접수")

    else:
        self.timer.stop()
        self.play_pause_button.setText("긴급상황\n접수 취소")
    self.is_paused = not self.is_paused

1 usage
def toggle_display(self):
    self.show_bbox = not self.show_bbox
    if self.show_bbox:
        self.toggle_button.setText("객체 BBOX\nOFF")
    else:
        self.toggle_button.setText("객체 BBOX\nON")

    if self.last_frame is not None and self.last_results is not None:
        self.update_display(self.last_frame, self.last_results)
```

| 핵심소스코드(8)

PyQt 모니터링(정보 업데이트)

```
row_position = self.table_widget.rowCount()
self.table_widget.insertRow(row_position)
self.table_widget.setItem(row_position, column: 0, QTableWidgetItem(str(int(track_id))))
self.table_widget.setItem(row_position, column: 1, QTableWidgetItem(cls_name))
self.table_widget.setItem(row_position, column: 2, QTableWidgetItem(f'{conf:.2f}'))
self.table_widget.setItem(row_position, column: 3, QTableWidgetItem(f'({x}, {y})'))
```

```
# A 영역 테이블 업데이트
row_position = self.a_area_table_widget.rowCount()
self.a_area_table_widget.insertRow(row_position)
self.a_area_table_widget.setItem(row_position, column: 0, QTableWidgetItem(str(int(track_id))))
self.a_area_table_widget.setItem(row_position, column: 1, QTableWidgetItem(cls_name))
self.a_area_table_widget.setItem(row_position, column: 2, QTableWidgetItem(transformed_textA))
self.a_area_table_widget.setItem(row_position, column: 3, QTableWidgetItem(str(self.av)))
```

```
# B 영역 테이블 업데이트
row_position = self.b_area_table_widget.rowCount()
self.b_area_table_widget.insertRow(row_position)
self.b_area_table_widget.setItem(row_position, column: 0, QTableWidgetItem(str(int(track_id))))
self.b_area_table_widget.setItem(row_position, column: 1, QTableWidgetItem(cls_name))
self.b_area_table_widget.setItem(row_position, column: 2, QTableWidgetItem(transformed_textB))
self.b_area_table_widget.setItem(row_position, column: 3, QTableWidgetItem(str(self.bv)))
```

| 핵심소스코드(9)

PyQt 모니터링(위험도)

```
# 충돌 예측
if self.current_a_position and self.current_b_position is not None:
    if self.current_a_position >= 450 or self.current_b_position >= 450:
        collision_status = self.evaluate_collision_risk()
        self.collision_label.setText(f"위험도: {collision_status}")

        if collision_status == "위험":
            self.broadcast_message(collision_status + "\n")
            print(datetime.datetime.now(), collision_status)
        elif collision_status == "주의":
            self.broadcast_message(collision_status + "\n")
            print(datetime.datetime.now(), collision_status)
        else:
            self.broadcast_message(collision_status + "\n")
            print(datetime.datetime.now(), collision_status)

    elif (self.current_a_position is None) != (self.current_b_position is None):
        collision_status = "주의"
        self.broadcast_message(collision_status + "\n")
        print(datetime.datetime.now(), collision_status)
    else:
        collision_status = "안전"
        self.broadcast_message(collision_status + "\n")
        print(datetime.datetime.now(), collision_status)
else:
    self.broadcast_message(collision_status + "\n")
    print(datetime.datetime.now(), collision_status)
    self.collision_label.setText(f"위험도: {collision_status}")
```

| 핵심소스코드(10)

아두이노(위험 상황 경보기)

```
void handleServerMessage(String message) {
  if (message.indexOf("안전") >= 0) {
    setLEDColor(strip.Color(0, 255, 0)); // 초록색
    noTone(BUZZER_PIN); // 부저 OFF
  } else if (message.indexOf("주의") >= 0) {
    setLEDColor(strip.Color(255, 255, 0)); // 노랑색
    noTone(BUZZER_PIN); // 부저 OFF
  } else if (message.indexOf("위험") >= 0) {
    setLEDColor(strip.Color(255, 0, 0)); // 빨간색
    tone(BUZZER_PIN, 500); // 부저 큰 소리
  }
}
```

| 핵심소스코드(11)

자율주행차량(차선 인식 주행)

```
try:
    while True:
        start = time.time()
        # Capture RGB Image from CSI
        myCam.read()

        croppedRGB = myCam.imageData[524:674, 0:820]
        hsvBuf = cv2.cvtColor(croppedRGB, cv2.COLOR_BGR2HSV)
        binaryImage = ImageProcessing.binary_thresholding(frame=_hsvBuf,lowerBounds=np.array([0, 0, 200]),upperBounds=np.array([180, 30, 255]))

        # Display the RGB (Original) as well as the Binary in 1/4th resolution for speed
        # cv2.imshow('My RGB image', cv2.resize(myCam.image_data, (410, 205) ) )
        # cv2.imshow('My RGB image', cropped_rgb )
        cv2.imshow( winname: 'My Binary image', cv2.resize(binaryImage,  dsize: (410, 75) ))
        slope, intercept = ImageProcessing.find_slope_intercept_from_binary(binary=binaryImage)

        rawSteering = 1.5*(slope - 0.3419) + (1/150)*(intercept+5)
        steering = steeringFilter.send((np.clip(rawSteering,  a_min: 0.5, -0.5), dt))

        # Write steering to qcar
        if not math.isnan(steering):
            throttle = 0.1  # 기본적인 전진 속도
            throttle *= np.cos(steering)
            steering_value = steering
        else:
            throttle = 0
            steering_value = 0

        LEDs = np.array([0, 0, 0, 0, 0, 0, 1, 1])
        myCar.read_write_std(throttle, steering_value, LEDs)
        cv2.waitKey(1)
        end = time.time()
        dt = end - start
except KeyboardInterrupt:
    print("User interrupted!")

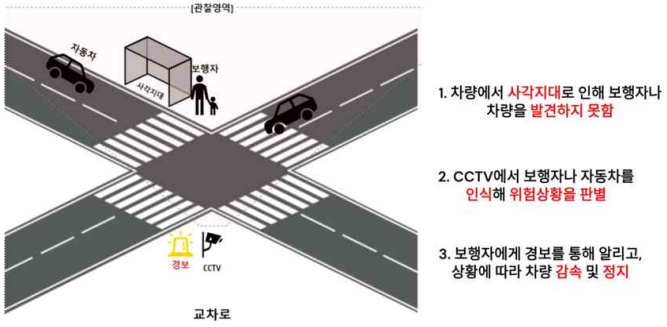
finally:
    myCam.terminate()
    myCar.terminate()
```

| 참조-동영상 촬영 콘티

1. 프로젝트 인트로
- 어떤 프로젝트인지 알 수 있도록 샘플영상과 간단 설명



2. 시스템 구조도
- 교통 사고 방지 시스템의 작동 구조를 설명



시스템 구조도

3. 커스텀 데이터 제작과 전처리 과정
- 사고/비사고 영상을 모아 객체 인식 및 추적에 필요한 데이터 축적



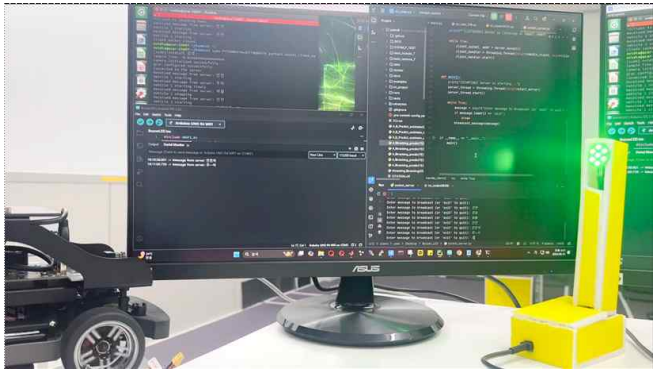
4. 각 모듈별 구현 기능

4-1. 자율주행 차량
- 사고 방지를 위한 차량제어 및 통신의 시연영상

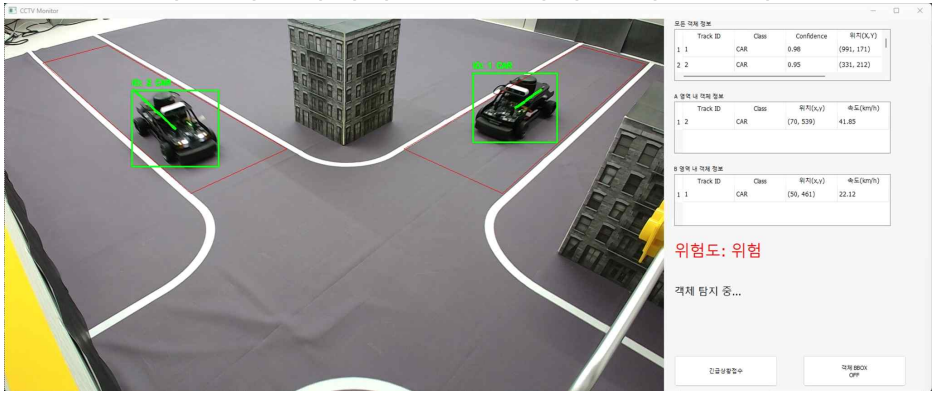


| 참조-동영상 촬영 콘티

4-2. 경보기 제어, 하드웨어 통신
- 제작한 하드웨어의 기능을 시연



4-3. 모니터링 화면
GUI를 통해 관제센터에서 보는 모니터링 화면을 시연



5. 예상 시나리오

5-1 시나리오1
- 차 vs 차 상황 판단시나리오

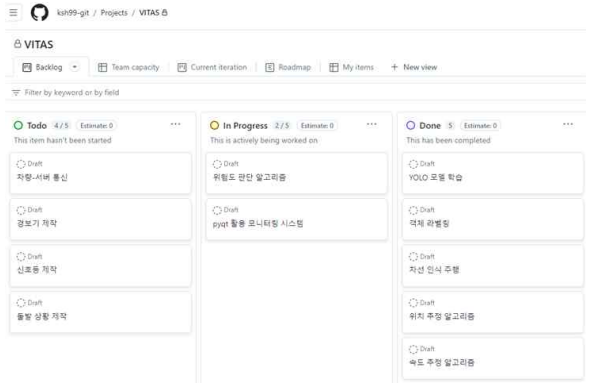


5-2 시나리오2
- 차 vs 사람 상황 판단 시나리오



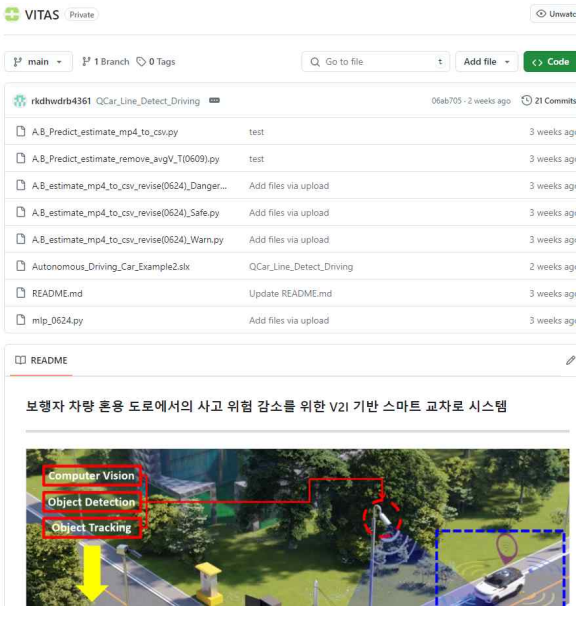
| 참조-프로젝트 관리

프로젝트 관리



git 활용

형상관리



git 활용

이슈관리

속도 및 위치값 추정 알고리즘 활용 특징 추출 #4

Closed ksh99-git opened this issue 3 weeks ago · 7 comments

ksh99-git commented 3 weeks ago • edited

A_slope	B_slope	distCA	distCB	Ax	Bx	situation
0	12.2	720	7	0	58	0
13.29	12.2	13	-183	80	58	0
0	7.53	720	2	0	63	0
13.14	7.53	3	-191	81	63	0
13.29	6.5	1	490	97	46	0

X 값: 속도(A_slope, B_slope), 위치(distCA, distCB, Ax, Bx)

Y 값: 위험도(situation)

하나의 비디오 영상에서 특징값은 총 2번 추출된다.
A영역(주 도로)의 객체가 A영역을 벗어날 때(교차로에 진입할 때)와 B영역(보조 도로)의 객체가 B영역을 벗어날 때(교차로에 진입할 때)이다.
각 위험도별로 72개의 영상을 선정했고 위험도는 총 3가지이다.
따라서 csv파일의 총 행의 수는 72(영상 수)*3(클래스 수)*2(추출 횟수)=432 이다.

git 활용

Thank you

